Kineo Internal Documentation

1. Create an Account Page

```
import { Colors } from '@/app-example/constants/theme';
import { useNavigation, useRouter } from 'expo-router';
import { useEffect, useState } from 'react';
import { Image, StyleSheet, Text, TextInput, TouchableOpacity, View, Alert } from
'react-native'; // Alert added
import MaterialIcons from '@expo/vector-icons/MaterialIcons';
import { auth } from '../../../configs/FirebaseConfig';
import { createUserWithEmailAndPassword } from 'firebase/auth';
import { BlurView } from 'expo-blur';

export default function Sign() {
 const navigation = useNavigation();
 const router = useRouter();

 const [email, setEmail] = useState('');
 const [password, setPassword] = useState('');
 const [fullName, setFullName] = useState('');

 useEffect(() => {
   navigation.setOptions({ headerShown: false });
 }, [navigation]);

 const OnCreateAccount = () => {

   // Check if any of the fields are empty
   if (!email || !password || !fullName) {
       Alert.alert('Error', 'Please Enter All Details');
       return;
   }
```

```
  console.log('Attempting signup with:', email, '| password length:',
password.length);

  createUserWithEmailAndPassword(auth, email, password)
    .then((userCredential) => {
      // Signed up
      const user = userCredential.user;
      console.log('User created:', user.uid);
      router.replace('/mytrip'); // navigate to MyTrip after successful signup
    })
    .catch((error) => {
      const errorCode = error.code;
      const errorMessage = error.message;
      console.log('Error signing up:', errorCode, errorMessage);
      // Show an alert for signup errors
      Alert.alert('Error', 'Failed to create account. ' + errorMessage);
    });
};

return (
  <View style={styles.mainContainer}>

    <View style={styles.headerWrapper}>
      <View style={styles.headerBack} />
      <View style={styles.header} />
    </View>

    <Image
      source={require('../../../assets/images/login_page/Logo_Nature.png')}
      style={styles.largeLogo}
    />

    {/* Content */}
    <View style={styles.content}>
      <Text style={styles.title}>KINEO</Text>
      <Text style={styles.subtitle}>Create an Account</Text>
      <TouchableOpacity onPress={() => router.back()}>
        <MaterialIcons name="arrow-back-ios" size={24} color="#0165bcff"/>
      </TouchableOpacity>
      <Text style={styles.welcome}>Welcome!</Text>

      <View style={styles.inputContainer}>
```

```
        {/* FULL NAME */}
        <Text style={styles.fullName}>Full Name</Text>
        <TextInput
          style={styles.input}
          placeholder="Enter Full Name"
          placeholderTextColor="#aaa"
          keyboardType="email-address"
          autoCapitalize="none"
          onChangeText={(value)=> setFullName(value)}
        />

        {/* EMAIL */}
        <Text style={styles.email}>Email</Text>
        <TextInput
          style={styles.input}
          placeholder="Enter Email"
          placeholderTextColor="#aaa"
          keyboardType="email-address"
          autoCapitalize="none"
          value={email}
          onChangeText={(value)=> setEmail(value)}
        />

        {/* PASSWORD */}
        <Text style={styles.password}>Password</Text>
        <TextInput
          secureTextEntry
          value={password}
          onChangeText={(value)=> setPassword(value)}
          style={styles.inputPassWord}
          placeholder="Enter Password"
          placeholderTextColor="#aaa"
          autoCapitalize="none"
        />

        {/* SIGN IN */}
        <View style = {styles.glassWrapperSignIn}>
        <TouchableOpacity
          onPress={() => router.replace('auth/sign-in')}
          style={styles.signInButton}
        >
          <Text style={styles.signIn}>Sign In</Text>
```

```
        </TouchableOpacity>
        </View>

          {/* CREATE ACCOUNT */}
          <View style = {styles.glassWrapperCreateAccount}>
          <BlurView intensity={60} tint="light" style={styles.searchContainerGlass}>
          <TouchableOpacity
            onPress={OnCreateAccount}
            style={styles.createAccountButton}
          >
            <Text style={styles.createAccount}>Create Account</Text>
          </TouchableOpacity>
          </BlurView>
        </View>

      </View>
    </View>
  </View>
 );
}

const styles = StyleSheet.create({
 mainContainer: {
   flex: 1,
   backgroundColor: '#f5f5f5', // Light gray-white background
   height: '29%',
   zIndex: 1,
 },
  headerWrapper: {//HEADER
   position: 'absolute',
   top: -260,
   left: -100,
   right: -100,
   height: 525,
   overflow: 'hidden',
   zIndex: 1
 },

 header: { //blue semi circle
   width: '100%',
   height: '93%',
   backgroundColor: Colors.BLUE,
```

```
  borderBottomLeftRadius: 320,
  borderBottomRightRadius: 320,
  zIndex: 2
},

headerBack: { //light blue semi circle
  position: 'absolute',
  top: 50,
  width: '100%',
  height: '90%',
  backgroundColor: Colors.BLUE,
  opacity: 0.2,
  borderBottomLeftRadius: 320,
  borderBottomRightRadius: 320,
  zIndex: 2,
},

/* ================ LOGO ================ */
largeLogo: {
  position: 'absolute',
  top: -90,
  right: -30,
  width: 380,
  height: 380,
  resizeMode: 'contain',
  opacity: 0.9,
  zIndex: 3,
  transform: [{ rotate: '-20deg' }], // rotates clockwise
},

content: {
  paddingHorizontal: 28,
  paddingTop: 140,
},

title: { // KINEO
  color: Colors.WHITE,
  fontFamily: 'outfit-bold',
  fontSize: 96,
  top: -80,
  textAlign: 'center',
  zIndex: 3,
```

```
},

subtitle: { // CREATE A NEW ACCOUNT
  color: Colors.WHITE,
  fontFamily: 'outfit-medium',
  fontSize: 26,
  textAlign: 'center',
  top: -72,
  marginTop: -10,
  zIndex: 3,
},

welcome: { // WELCOME!
  color: Colors.BLUE,
  fontFamily: 'outfit-medium',
  fontSize: 25,
  marginTop: 20,
},

inputContainer: {
  marginTop: 30,
},

fullName: { // FULL NAME
  color: '#000',
  fontFamily: 'outfit-medium',
  fontSize: 16,
  marginLeft: 5,
  marginBottom: 8,
},

input: {
  backgroundColor:  Colors.WHITE,
  borderRadius: 30,
  paddingHorizontal: 20,
  paddingVertical: 16,
  fontSize: 16,
  shadowColor: Colors.BLUE,
  shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.1,
  shadowRadius: 4,
  elevation: 3,
```

```
},

email: { // EMAIL TEXT
  color: '#000',
  fontFamily: 'outfit-medium',
  paddingTop: 20,
  fontSize: 16,
  marginLeft: 5,
  marginBottom: 8,
},

password: { // PASSWORD TEXT
  color: '#000',
  fontFamily: 'outfit-medium',
  paddingTop: 20,
  fontSize: 16,
  marginLeft: 5,
  marginBottom: 6,
},

inputPassWord: {
  backgroundColor: Colors.WHITE,
  marginTop: 5,
  borderRadius: 30,
  paddingHorizontal: 20,
  paddingVertical: 16,
  fontSize: 16,
  shadowColor: Colors.BLUE,
  shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.1,
  shadowRadius: 4,
  elevation: 3,
},

/* ================= BUTTONS ================= */
signInButton: {
  padding: 15,
  backgroundColor: Colors.BLUE,
  borderRadius: 99,
  borderColor: "rgba(213, 234, 255, 1)"
},
```

```
 signIn: { // SIGN IN BUTTON
   color: Colors.WHITE,
   textAlign: 'center',
   fontFamily: 'outfit-medium',
   fontSize: 16,
 },

 createAccountButton: {
   padding: 15,
   backgroundColor: 'transparent', // CHANGED TO TRANSPARENT
   borderRadius: 99,
 },

 createAccount: { // CREATE ACCOUNT BUTTON
   color: Colors.BLUE,
   textAlign: 'center',
   fontFamily: 'outfit-medium',
   fontSize: 16,
 },

 glassWrapperSignIn: {
   marginTop: 30,
   borderRadius: 99,
   overflow: 'hidden',
   borderWidth: 5,
   borderColor: 'rgba(35, 137, 255, 0.1)',
 },

 glassWrapperCreateAccount: {
   marginTop: 30,
   borderRadius: 99,
   overflow: 'hidden',
   borderWidth: 5,
   borderColor: 'rgba(255, 255, 255, 0.8)',
 },

 searchContainerGlass: {
   width: '100%',
   backgroundColor: 'rgba(255, 255, 255, 0.4)',
 },

});
```

2. Sign In Page

```
import { Colors } from "@/app-example/constants/theme";
import MaterialIcons from "@expo/vector-icons/MaterialIcons";
import { useNavigation, useRouter } from "expo-router";
import { signInWithEmailAndPassword } from "firebase/auth";
import { useEffect, useState } from "react";
import {
 Alert,
 Image,
 StyleSheet,
 Text,
 TextInput,
 TouchableOpacity,
 View,
} from "react-native";

import { auth } from "../../../configs/FirebaseConfig";

export default function Sign() {
 const navigation = useNavigation();
 const router = useRouter();

 const [email, setEmail] = useState("");
 const [password, setPassword] = useState("");
 const [loading, setLoading] = useState(false);

 useEffect(() => {
   navigation.setOptions({ headerShown: false });
 }, []);

 const onSignIn = async () => {
   if (!email || !password) {
     Alert.alert("Error", "Please enter email and password");
     return;
   }

   try {
     setLoading(true);
```

```
    const userCredential = await signInWithEmailAndPassword(
      auth,
      email.trim(),
      password.trim()
    );

    console.log("Signed in:", userCredential.user.uid);

    router.replace("/mytrip");
  } catch (error) {
    console.log("Firebase error:", error.code);

    if (error.code === "auth/invalid-email") {
      Alert.alert("Invalid Email", "Please enter a valid email.");
    } else if (error.code === "auth/user-not-found") {
      Alert.alert("User Not Found", "No account exists with this email.");
    } else if (error.code === "auth/wrong-password") {
      Alert.alert("Wrong Password", "Incorrect password.");
    } else if (error.code === "auth/invalid-credential") {
      Alert.alert("Invalid Credentials", "Email or password is incorrect.");
    } else {
      Alert.alert("Login Failed", "Something went wrong.");
    }
  } finally {
    setLoading(false);
  }
};

return (
  <View style={styles.mainContainer}>
    <View style={styles.headerWrapper}>
      <View style={styles.headerBack} />
      <View style={styles.header} />
    </View>

    <Image
      source={require("../../../assets/images/login_page/Logo_Nature.png")}
      style={styles.largeLogo}
    />

    <View style={styles.content}>
      <Text style={styles.title}>KINEO</Text>
```

```
      <Text style={styles.subtitle}>Let&apos;s Sign You In</Text>

      <TouchableOpacity onPress={() => router.back()}>
        <MaterialIcons name="arrow-back-ios" size={24} color="#0165bcff" />
      </TouchableOpacity>

      <Text style={styles.welcome}>Ready To Get Travellin&apos;?</Text>

      <View style={styles.inputContainer}>
        {/* EMAIL */}
        <Text style={styles.label}>Email</Text>

        <TextInput
          value={email}
          style={styles.input}
          onChangeText={setEmail}
          placeholder="Enter Email"
          placeholderTextColor="#aaa"
          keyboardType="email-address"
          autoCapitalize="none"
        />

        {/* PASSWORD */}

        <Text style={styles.password}>Password</Text>

        <TextInput
          value={password}
          secureTextEntry
          style={styles.inputPassWord}
          onChangeText={setPassword}
          placeholder="Enter Password"
          placeholderTextColor="#aaa"
          autoCapitalize="none"
        />

        {/* SIGN IN */}
        <TouchableOpacity
          onPress={onSignIn}
          style={styles.signInButton}
          disabled={loading}
        >
```

```
          <Text style={styles.signIn}>
            {loading ? "Signing In..." : "Sign In"}
          </Text>
        </TouchableOpacity>

        {/* CREATE ACCOUNT */}

        <TouchableOpacity
          onPress={() => router.push("/auth/create-account")}
          style={styles.createAccountButton}
        >
          <Text style={styles.createAccount}>Create Account</Text>
        </TouchableOpacity>

        {/* CONTINUE AS GUEST */}

        <TouchableOpacity onPress={() => router.replace("/discover")}>
          <Text style={styles.withoutSignIn}>Continue as Guest</Text>
        </TouchableOpacity>
      </View>
    </View>
  </View>
 );
}

const styles = StyleSheet.create({
 mainContainer: {
   flex: 1,
   backgroundColor: "#f5f5f5",
 },

 headerWrapper: {
   position: "absolute",
   top: -260,
   left: -100,
   right: -100,
   height: 525,
   overflow: "hidden",
   zIndex: 1,
 },

 header: {
```

```
  width: "100%",
  height: "93%",
  backgroundColor: Colors.BLUE,
  borderBottomLeftRadius: 320,
  borderBottomRightRadius: 320,
  zIndex: 2,
},

headerBack: {
  position: "absolute",
  top: 50,
  width: "100%",
  height: "90%",
  backgroundColor: Colors.BLUE,
  opacity: 0.2,
  borderBottomLeftRadius: 320,
  borderBottomRightRadius: 320,
},

largeLogo: {
  position: "absolute",
  top: -90,
  right: -30,
  width: 380,
  height: 380,
  resizeMode: "contain",
  opacity: 0.9,
  zIndex: 3,
  transform: [{ rotate: "-20deg" }],
},

content: {
  paddingHorizontal: 28,
  paddingTop: 140,
},

title: {
  color: Colors.WHITE,
  fontFamily: "outfit-bold",
  fontSize: 96,
  top: -80,
  textAlign: "center",
```

```
},

subtitle: {
  color: Colors.WHITE,
  fontFamily: "outfit-medium",
  fontSize: 26,
  textAlign: "center",
  top: -72,
},

welcome: {
  color: Colors.BLUE,
  fontFamily: "outfit-medium",
  fontSize: 25,
  marginTop: 25,
},

inputContainer: {
  marginTop: 40,
},

label: {
  color: "#000",
  fontFamily: "outfit-medium",
  fontSize: 16,
  marginLeft: 5,
  marginBottom: 8,
},

password: {
  color: "#000",
  fontFamily: "outfit-medium",
  fontSize: 16,
  marginTop: 15,
  marginLeft: 5,
  marginBottom: 3,
},

input: {
  backgroundColor: Colors.WHITE,
  borderRadius: 30,
  paddingHorizontal: 20,
```

```
  paddingVertical: 16,
  fontSize: 16,
  elevation: 3,
},

inputPassWord: {
  backgroundColor: Colors.WHITE,
  marginTop: 5,
  borderRadius: 30,
  paddingHorizontal: 20,
  paddingVertical: 16,
  fontSize: 16,
  elevation: 3,
},

signInButton: {
  padding: 15,
  backgroundColor: Colors.BLUE,
  borderRadius: 99,
  marginTop: 50,
},

signIn: {
  color: Colors.WHITE,
  textAlign: "center",
  fontFamily: "outfit-medium",
  fontSize: 16,
},

createAccountButton: {
  padding: 15,
  backgroundColor: Colors.WHITE,
  borderRadius: 99,
  marginTop: 30,
  borderWidth: 1,
  borderColor: Colors.BLUE,
},

createAccount: {
  color: Colors.BLUE,
  textAlign: "center",
  fontFamily: "outfit-medium",
```

```
    fontSize: 16,
  },

  withoutSignIn: {
    color: Colors.BLUE,
    fontFamily: "outfit-medium",
    textAlign: "center",
    fontSize: 16,
    marginTop: 15,
  },
});
```

3. Explore Page

```
import { Colors } from "@/app-example/constants/theme";
import EvilIcons from "@expo/vector-icons/EvilIcons";
import FontAwesome from "@expo/vector-icons/FontAwesome";
import { LinearGradient } from "expo-linear-gradient";
import { useNavigation, useRouter } from "expo-router";
import { useContext, useEffect, useRef, useState } from "react";
import {
  ActivityIndicator,
  Image,
  ImageBackground,
  StyleSheet,
  Text,
  TouchableOpacity,
  View,
} from "react-native";
import Swiper from "react-native-deck-swiper";
import { GooglePlacesAutocomplete } from "react-native-google-places-autocomplete";
import { defaultPlaces } from "../../app-example/constants/RandomPlaces";
import { chatSession } from "../../configs/RandomPlacesAi";
import { CreateTripContext } from "../../context/CreateTripsContext";
import { LikedTripsContext } from "../../context/LikedTripsContext";

// CARD WITH BUILT-IN LOADING STATE PER IMAGE
const DestinationCard = ({ card }) => {
  const [loaded, setLoaded] = useState(false);

  const Subscription = ({ card }) => {
    const [subscribed, setSubscribed] = useState(false);
```

```
 }
 return (
   <View style={styles.card}>
     {/* SHIMMER PLACEHOLDER SHOWN WHILE IMAGE LOADS */}
     {!loaded && (
       <View style={styles.placeholder}>
         <ActivityIndicator size="large" color="#0B5ED7" />
       </View>
     )}
     <Image
     source={card.placeImage ? card.placeImage : { uri: card.placeImageUrl }}
     style={[styles.cardImage, !loaded && { opacity: 0 }]}
     onLoad={() => setLoaded(true)}
     fadeDuration={300}
     />
     <LinearGradient
       colors={["transparent", "rgba(0,0,0,0.8)"]}
       style={styles.cardInfo}
     >
       <Text style={styles.cardTitle}>{card.placeName}</Text>
       <Text style={styles.cardLocation}>
         {card.capitalCity}, {card.country}
       </Text>
     </LinearGradient>
   </View>
 );
};

export default function Discover() {
 const navigation = useNavigation();
 const router = useRouter();
 const swiperRef = useRef(null);

 const { setTripData } = useContext(CreateTripContext);
 const { addLikedTrip } = useContext(LikedTripsContext);

 const [places, setPlaces] = useState(defaultPlaces);
 const [swiperKey, setSwiperKey] = useState(0);

 useEffect(() => {
   navigation.setOptions({
     headerShown: false,
```

```
    headerTransparent: true,
  });
}, []);

const getRandomPlaces = async () => {
  try {
    const result = await chatSession.sendMessage(
      "Return ONLY valid JSON array of 10 RANDOM famous travel places. Make sure
results are different each time. Include placeName, placeImageUrl, description,
capitalCity, country.",
    );

    const text = await result.response.text();
    const cleaned = text
      .replace(/```json/g, "")
      .replace(/```/g, "")
      .trim();

    const parsedData = JSON.parse(cleaned);

    if (Array.isArray(parsedData)) {
      setPlaces(parsedData);
      setSwiperKey((prev) => prev + 1);
      setTimeout(() => {
        swiperRef.current?.jumpToCardIndex(0);
      }, 100);
      console.log("Destinations Refreshed");
    } else {
      console.error("AI response is not array:", parsedData);
    }
  } catch (error) {
    console.error("Gemini error:", error);
  }
};

const handleLike = async (index) => {
  if (!places[index]) return;
  await addLikedTrip(places[index]);
  console.log("Liked:", places[index].placeName);
};

const handleDislike = (index) => {
```

```
  if (!places[index]) return;
  console.log("Passed:", places[index].placeName);
};

return (
  <View style={{ flex: 1, backgroundColor: "#fff" }}>
    {/* HEADER IMAGE + SEARCH */}
    <ImageBackground
      source={require("./../../assets/images/login_page/Explore-top-picture.png")}
      style={styles.imageContainer}
      imageStyle={{
        borderBottomLeftRadius: 25,
        borderBottomRightRadius: 25,
      }}
    >
      <View style={{ top: 80 }}>
        <View style={styles.searchContainer}>
          <FontAwesome name="search" size={20} color="#fff" />
          <GooglePlacesAutocomplete
            placeholder="Search"
            placeholderTextColor="#fff"
            fetchDetails={true}
            keyboardShouldPersistTaps="handled"
            onPress={(data, details = null) => {
              setTripData({
                locationInfo: {
                  name: data.description,
                  coordinates: details?.geometry.location || null,
                  photoRef: details?.photos?.[0]?.photo_reference || null,
                  url: details?.url || null,
                },
              });
              router.push("../create-trip/travelInformation");
            }}
            query={{
              key: process.env.EXPO_PUBLIC_GOOGLE_MAP_KEY,
              language: "en",
            }}
            listViewDisplayed="auto"
            debounce={400}
            enablePoweredByContainer={false}
            minLength={2}
```

```
          styles={{
            container: { flex: 1, zIndex: 9999 },
            textInputContainer: {
              flex: 1,
              backgroundColor: "transparent",
              height: 50,
            },
            textInput: {
              fontSize: 18,
              fontFamily: "outfit",
              color: "#fff",
              paddingLeft: 10,
              backgroundColor: "transparent",
            },
            listView: {
              position: "absolute",
              backgroundColor: "#fff",
              borderRadius: 12,
              marginTop: 10,
              borderWidth: 1,
              borderColor: "#e0e0e0",
              maxHeight: 360,
              top: 45,
              left: -35,
              right: -15,
              zIndex: 999,
              elevation: 8,
            },
            row: { padding: 13, backgroundColor: "#fff" },
            separator: { backgroundColor: "#e0e0e0" },
          }}
        />
      </View>
    </View>
  </ImageBackground>

  {/* TITLE + REFRESH */}
  <View style={styles.title}>
    <Text style={styles.popular}>Popular Destinations</Text>
    <TouchableOpacity onPress={getRandomPlaces}>
      <EvilIcons
        name="refresh"
```

```
          size={40}
          color="#0B5ED7"
          style={styles.refresh}
        />
      </TouchableOpacity>
    </View>

    {/* SWIPER */}
    <View style={styles.swiperContainer}>
      <Swiper
        key={swiperKey}
        ref={swiperRef}
        cards={places}
        renderCard={(card) => {
          if (!card) return null;
          return <DestinationCard card={card} />;
        }}
        onSwipedLeft={handleDislike}
        onSwipedRight={handleLike}
        backgroundColor="transparent"
        stackSize={10}
        stackSeparation={5}
        stackScale={0.92}
        overlayLabels={{
          left: {
            title: "I'll Pass!",
            style: {
              label: {
                fontFamily: "outfit-bold",
                color: "#fff",
                fontSize: 30,
              },
              wrapper: {
                flex: 1,
                backgroundColor: "rgba(249, 0, 0, 0.35)",
                position: "absolute",
                justifyContent: "center",
                alignItems: "center",
                borderRadius: 20,
                height: 460,
                left: -5,
              },
```

```
            },
          },
          right: {
            title: "I want to go here!",
            style: {
              label: {
                fontFamily: "outfit-bold",
                color: "#fff",
                fontSize: 30,
              },
              wrapper: {
                flex: 1,
                backgroundColor: "rgba(41, 250, 4, 0.36)",
                position: "absolute",
                justifyContent: "center",
                alignItems: "center",
                borderRadius: 20,
                height: 460,
                right: -5
              },
            },
          },
        }}
        cardVerticalMargin={25}
        useViewOverflow
        showSecondCard={true}
      />
    </View>
  </View>
 );
}

const styles = StyleSheet.create({
 imageContainer: {
   paddingTop: 50,
   paddingHorizontal: 20,
   paddingBottom: 100,
 },

 searchContainer: {
   flexDirection: "row",
   alignItems: "center",
```

```
  borderRadius: 20,
  borderWidth: 2,
  borderColor: "#fff",
  backgroundColor: "rgba(71, 71, 71, 0.15)",
  paddingHorizontal: 15,
  height: 50,
},

title: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  paddingHorizontal: 20,
  marginTop: 15,
},

popular: {
  fontSize: 25,
  color: Colors.BLUE,
  fontFamily: "outfit-bold",
  marginTop: 5,
},

refresh: {
  top: 10,
},

swiperContainer: {
  alignItems: "center",
  marginVertical: 20,
},

card: {
  width: 350,
  borderRadius: 20,
  overflow: "hidden",
  marginLeft: 5,
},

// GRAY PLACEHOLDER SHOWN WHILE IMAGE IS LOADING
placeholder: {
  position: "absolute",
```

```
    width: "100%",
    height: 460,
    backgroundColor: "#d0d0d0",
    justifyContent: "center",
    alignItems: "center",
    borderRadius: 20,
  },

  cardImage: {
    width: "100%",
    height: 460,
  },

  cardInfo: {
    position: "absolute",
    bottom: 0,
    padding: 20,
    width: "100%",
  },

  cardTitle: {
    fontSize: 28,
    color: "#fff",
    fontWeight: "bold",
  },

  cardLocation: {
    fontSize: 18,
    color: "#fff",
  },
});
```

4. My Trips Page

```
import { Colors } from "@/app-example/constants/theme";
import { useRouter } from "expo-router";
import { collection, getDocs, query, where } from "firebase/firestore";
import { useEffect, useState } from "react";
import {
  ActivityIndicator,
  ScrollView,
  StyleSheet,
```

```
 Text,
 TouchableOpacity,
 View,
} from "react-native";
import StartNewTripsCard from "../../components/MyTrips/StartNewTripsCard";
import UserTripList from "../../components/MyTrips/UserTripList";
import LikedTrips from "../../components/MyTrips/LikedTrips";
import { auth, db } from "./../../../configs/FirebaseConfig";

export default function MyTrip() {
 const router = useRouter();
 const [userTrips, setUserTrips] = useState([]);
 const [loading, setIsLoading] = useState(false);

 // TRACKS WHICH TAB IS CURRENTLY ACTIVE - "ongoing" OR "liked"
 const [activeTab, setActiveTab] = useState("ongoing"); // FIX: was written as
useState=("ongoing") which is wrong syntax

 const user = auth.currentUser;

 // GETUSERTRIPS WILL NOT RUN UNTIL THERE IS A USER LOGGED IN
 useEffect(() => {
   user && GetMyTrips();
 }, [user]); // FIX: removed the extra stray semicolon after the closing bracket

 // ASYNC AND AWAIT MAKES THE CODE BEHAVE MORE SYNCHRONOUSLY
 // WAITING FOR EACH STEP BEFORE STARTING THE NEXT, WHICH MAKES IT
 // READABLE AND PREVENTS UI FREEZING

 // FETCHES ALL TRIPS BELONGING TO THE CURRENTLY LOGGED IN USER
 const GetMyTrips = async () => {
   setIsLoading(true);

   // RESET TRIPS LIST WHENEVER LOADING NEW DATA TO PREVENT STALE ENTRIES
   setUserTrips([]);

   // QUERY THE DATABASE FOR DOCUMENTS WHERE userEmail MATCHES THE CURRENT USER
   const q = query(
     collection(db, "NewUserTrips"),
     where("userEmail", "==", user?.email)
   );
```

```
  // PAUSE HERE UNTIL ALL MATCHING DOCUMENTS ARE RETRIEVED FROM FIRESTORE
  const querySnapshot = await getDocs(q);

  // LOOP THROUGH EACH DOCUMENT AND APPEND IT TO THE TRIPS STATE
  querySnapshot.forEach((doc) => {
    console.log(doc.id, " => ", doc.data());
    setUserTrips((prev) => [...prev, doc.data()]);
  });

  setIsLoading(false);
};

return (
  <ScrollView
    style={styles.containerScrollView}
    contentContainerStyle={{
      paddingBottom: 25,
      paddingTop: 75,
      backgroundColor: Colors.WHITE,
    }}
  >
    <View>
      {/* PAGE TITLE */}
      <Text style={styles.myTripsText}>My Trips</Text>

      {/* PILL TOGGLE — SWITCHES BETWEEN ONGOING TRIPS AND LIKED TRIPS */}
      <View style={styles.tabContainer}>

        {/* ONGOING TRIPS TAB — ACTIVE BY DEFAULT */}
        <TouchableOpacity
          style={[
            styles.tabButton,
            activeTab === "ongoing" && styles.tabActive,
          ]}
          onPress={() => setActiveTab("ongoing")}
          activeOpacity={0.85}
        >
          <Text
            style={[
              styles.tabText,
              activeTab === "ongoing" && styles.tabTextActive,
            ]}
```

```
          >
            On Going Trips
          </Text>
        </TouchableOpacity>

        {/* LIKED TRIPS TAB — OPENS THE LIKEDTRIPS COMPONENT WHEN PRESSED */}
        <TouchableOpacity
          style={[
            styles.tabButton,
            activeTab === "liked" && styles.tabActive,
          ]}
          onPress={() => setActiveTab("liked")}
          activeOpacity={0.85}
        >
          <Text
            style={[
              styles.tabText,
              activeTab === "liked" && styles.tabTextActive,
            ]}
          >
            Liked Trips
          </Text>
        </TouchableOpacity>
      </View>
    </View>

    {/* CONDITIONALLY RENDER CONTENT BASED ON WHICH TAB IS SELECTED */}
    {activeTab === "ongoing" ? (
      <>
        {/* SHOW LOADING SPINNER WHILE TRIPS ARE BEING FETCHED */}
        {loading && (
          <ActivityIndicator size={"large"} color={Colors.PRIMARY} />
        )}

        {/* IF NO TRIPS EXIST SHOW THE EMPTY STATE CARD, OTHERWISE SHOW THE TRIP LIST
*/}
        {userTrips?.length === 0 ? (
          <StartNewTripsCard />
        ) : (
          <UserTripList userTrips={userTrips} />
        )}
      </>
```

```
    ) : (
      // RENDER THE LIKED TRIPS SCREEN WHEN THE LIKED TAB IS ACTIVE
      <LikedTrips />
    )}
  </ScrollView>
);
}

const styles = StyleSheet.create({
 containerScrollView: {
   flex: 1,
   backgroundColor: "#fff",
 },

 myTripsText: {
   textAlign: "center",
   paddingTop: 0,
   padding: 15,
   fontFamily: "outfit-bold",
   fontSize: 30,
 },

 tabContainer: {
   flexDirection: "row",
   backgroundColor: "#EFEFEF",
   borderRadius: 50,
   marginHorizontal: 20,
   marginBottom: 16,
   padding: 4,
 },

 tabButton: {
   flex: 1,
   paddingVertical: 10,
   borderRadius: 50,
   alignItems: "center",
   justifyContent: "center",
 },

 tabActive: {
   backgroundColor: Colors.BLUE,
 },
```

```
 tabText: {
   fontFamily: "outfit-bold",
   fontSize: 15,
   color: "#888",
 },

 tabTextActive: {
   color: "#fff",
 },
});
```

5. Travel Details

```
import { Colors } from "@/app-example/constants/theme";
import Feather from "@expo/vector-icons/Feather";
import { useRouter, useLocalSearchParams } from "expo-router";
import moment from "moment";
import { useContext, useEffect, useState } from "react";
import {
 Alert,
 FlatList,
 ImageBackground,
 ScrollView,
 StyleSheet,
 Text,
 TouchableOpacity,
 View,
} from "react-native";
import CalendarPicker from "react-native-calendar-picker";
import { SelectBudgetList } from "../../app-example/constants/OptionsBudget";
import { SelectPeopleList } from "../../app-example/constants/OptionsPeople";
import OptionCard from "../../components/CreateTrip/OptionCard";
import { CreateTripContext } from "../../context/CreateTripsContext";

export default function TravelInformation() {
 const router = useRouter();

 const [startDate, setStartDate] = useState();
 const [endDate, setEndDate] = useState();
 const [selectedOption, setSelectedOption] = useState();
 const [selectedBudget, setSelectedBudget] = useState();
```

```
const { tripData, setTripData } = useContext(CreateTripContext);

// receive destination from LikedTrips
const { trip } = useLocalSearchParams();
const place = trip ? JSON.parse(trip) : null;

// properly store destination in context
useEffect(() => {
  if (place) {
    setTripData((prev) => ({
      ...prev,
      locationInfo: {
        name: place.placeName,
        placeImageUrl: place.placeImageUrl,
        capitalCity: place.capitalCity,
        country: place.country,
      },
    }));
  }
}, [place]);

const onDateChange = (date, type) => {
  if (type === "START_DATE") {
    setStartDate(moment(date));
  } else {
    setEndDate(moment(date));
  }
};

const onDateSelectionContinue = () => {
  if (!startDate || !endDate) {
    Alert.alert("Missing Date", "Please select travel dates first.");
    return;
  }

  if (!selectedOption) {
    Alert.alert("Missing Travellers", "Please select travellers first.");
    return;
  }

  if (!selectedBudget) {
```

```
    Alert.alert("Missing Budget", "Please select a budget option.");
    return;
  }

  const totalNoOfDays = endDate.diff(startDate, "days");

  // SAFE MERGE (prevents losing destination + other data)
  setTripData((prev) => ({
    ...prev,
    startDate,
    endDate,
    totalNoOfDays: totalNoOfDays + 1,
    travellers: selectedOption,
    budget: selectedBudget,
  }));

  router.push("../create-trip/review-trip");
};

return (
  <ScrollView
    style={styles.containerScrollView}
    contentContainerStyle={{ paddingBottom: 40 }}
  >
    <View style={{ flex: 1 }}>

      {/* HEADER IMAGE */}
      <ImageBackground
        source={require("../../assets/images/TravelInformation/TravelDetails.png")}
        style={styles.travelDetailsContainer}
        resizeMode="cover"
      >
        <TouchableOpacity onPress={() => router.back()}>
          <Feather
            name="arrow-left"
            size={30}
            color="#ffffffff"
            style={styles.backButton}
          />
        </TouchableOpacity>
      </ImageBackground>
```

```
      {/* CALENDAR TITLE */}
      <Text style={styles.date}>Calendar</Text>

      {/* CALENDAR */}
      <View style={styles.calendarPicker}>
        <CalendarPicker
          onDateChange={onDateChange}
          allowRangeSelection={true}
          minDate={new Date()}
          selectedRangeStyle={{ backgroundColor: Colors.BLUE }}
          selectedDayTextStyle={{ color: Colors.WHITE }}
          todayBackgroundColor={Colors.BLUE}
          selectedDayColor={Colors.BLUE}
          selectedDayTextColor="#FFFFFF"
          textStyle={styles.calendarText}
          todayTextStyle={styles.todayText}
          monthTitleStyle={styles.monthTitleStyle}
          yearTitleStyle={styles.yearTitleStyle}
          weekdays={["S", "M", "T", "W", "T", "F", "S"]}
          previousTitle="←"
          nextTitle="→"
        />
      </View>

      {/* VENTURE SECTION (UNCHANGED DESIGN) */}
      <View style={styles.greyVentureContainer}>
        <Text style={styles.ventureWithText}>Venture With</Text>

        <FlatList
          data={SelectPeopleList}
          keyExtractor={(item) => item.id.toString()}
          numColumns={2}
          columnWrapperStyle={{ justifyContent: "space-between" }}
          contentContainerStyle={{ paddingHorizontal: 16, paddingBottom: 40 }}
          scrollEnabled={false}
          renderItem={({ item }) => (
            <OptionCard
              option={item}
              selectedOption={selectedOption}
              onPress={() => setSelectedOption(item)}
            />
          )}
```

```
        />

        <Text style={styles.ventureWithText}>Budget</Text>

        <FlatList
          data={SelectBudgetList}
          keyExtractor={(item) => item.id.toString()}
          numColumns={2}
          columnWrapperStyle={{ justifyContent: "space-between" }}
          contentContainerStyle={{ paddingHorizontal: 16, paddingBottom: 40 }}
          scrollEnabled={false}
          renderItem={({ item }) => (
            <OptionCard
              option={item}
              selectedOption={selectedBudget}
              onPress={() => setSelectedBudget(item)}
            />
          )}
        />
      </View>

      {/* CONTINUE BUTTON (UNCHANGED DESIGN) */}
      <TouchableOpacity
        onPress={onDateSelectionContinue}
        style={styles.continueButton}
      >
        <Text style={styles.continueText}>Continue</Text>
      </TouchableOpacity>

    </View>
  </ScrollView>
 );
}

const styles = StyleSheet.create({
 containerScrollView: {
   flex: 1,
   backgroundColor: "#fff",
 },

 ventureWithText: {
   fontFamily: "outfit-bold",
```

```
  color: Colors.BLUE,
  fontSize: 25,
  paddingLeft: 25,
  marginTop: 30,
  marginBottom: 5,
},

greyVentureContainer: {
  marginTop: 0,
  backgroundColor: "rgba(255, 255, 255, 0.2)",
  borderRadius: 20,
},

calendarPicker: {
  backgroundColor: "#fff",
  borderColor: "rgba(81, 81, 81, 0.08)",
  borderWidth: 2,
  borderRadius: 20,
  padding: 20,
  justifyContent: "center",
  marginHorizontal: 16,
  marginTop: 20,
  height: 350,
},

travelDetailsContainer: {
  height: 235,
  width: "100%",
  justifyContent: "center",
  alignItems: "center",
  overflow: "hidden",
  borderBottomLeftRadius: 25,
  borderBottomRightRadius: 25,
  right: 5,
},

backButton: {
  position: "absolute",
  right: 150,
  top: 0,
},
```

```
date: {
  fontFamily: "outfit-bold",
  color: Colors.BLUE,
  fontSize: 25,
  paddingLeft: 25,
  marginTop: 30,
},

continueButton: {
  backgroundColor: Colors.BLUE,
  borderRadius: 99,
  justifyContent: "center",
  alignItems: "center",
  height: 50,
  width: 250,
  alignSelf: "center",
  marginTop: 20,
},

continueText: {
  fontFamily: "outfit-medium",
  fontSize: 20,
  color: Colors.WHITE,
},

calendarText: {
  fontFamily: "outfit-regular",
  fontSize: 15,
},

todayText: {
  fontWeight: "bold",
},

monthTitleStyle: {
  fontFamily: "outfit-bold",
  fontSize: 18,
  color: Colors.BLUE,
},

yearTitleStyle: {
  fontFamily: "outfit-bold",
```

```
    fontSize: 18,
    color: Colors.BLUE,
  },
});
```

6. Review Trip

```
import { Colors } from "@/app-example/constants/theme";
import Feather from "@expo/vector-icons/Feather";
import FontAwesome from "@expo/vector-icons/FontAwesome";
import MaterialCommunityIcons from "@expo/vector-icons/MaterialCommunityIcons";
import Octicons from "@expo/vector-icons/Octicons";
import { BlurView } from "expo-blur";
import { useNavigation, useRouter } from "expo-router";
import { doc, setDoc } from "firebase/firestore";
import moment from "moment";
import { useContext, useEffect, useState } from "react";
import {
 ActivityIndicator,
 Alert,
 Image,
 Modal,
 StyleSheet,
 Text,
 TextInput,
 TouchableOpacity,
 View,
} from "react-native";
import { AI_PROMPT } from "../../app-example/constants/OptionsPeople";
import { chatSession } from "../../configs/AiModel";
import { CreateTripContext } from "../../context/CreateTripsContext";
import { auth, db } from "./../../../configs/FirebaseConfig";

export default function ReviewTrip() {
 const navigation = useNavigation();
 const router = useRouter();
 const { tripData } = useContext(CreateTripContext);

 //state for the loading block
 const [isLoading, setIsLoading] = useState(false);

 //get the email and pass authentication from firebase auth
 const user = auth.currentUser;
```

```
//cancel handle if taking too long to load
const handleCancel = () => {
  setIsLoading(false);
};

//Handle continue button
const handleContinue = () => {
  // Basic validation before showing loading
  if (!tripData?.locationInfo?.name) {
    Alert.alert("Missing Destination", "Please select a destination.");
    return;
  }
  if (!tripData?.travellers?.title) {
    Alert.alert("Missing Travellers", "Please select travellers first.");
    return;
  }
  if (!tripData?.startDate || !tripData?.endDate) {
    Alert.alert("Missing Date", "Please select travel dates first.");
    return;
  }
  if (!tripData?.budget?.title2) {
    Alert.alert("Missing Budget", "Please select a budget option.");
    return;
  }

  // loading animatio will show when continue to button is clicked
  setIsLoading(true);

  // now the method will activate
  GenerateAiTrip();
};

// the AI generating
const GenerateAiTrip = async () => {
  try {
    console.log("STARTING AI TRIP GENERATION");
    // Create the final prompt by replacing placeholders with user's trip data
    const FINAL_PROMPT = AI_PROMPT.replace(
      "{location}",
      tripData?.locationInfo?.name,
    )
```

```
    .replace("{totalDays}", tripData?.totalNoOfDays)
    .replace("{totalNights}", tripData?.totalNoOfDays - 1)
    .replace("{travellers}", tripData?.travellers?.title)
    .replace("{budget}", tripData?.budget?.title2)
    .replace("{totalDays}", tripData?.totalNoOfDays)
    .replace("{totalNights}", tripData?.totalNoOfDays - 1);

  // sending the prompt to gemini ai by calling the chatsession in AiModel
  const result = await chatSession.sendMessage(FINAL_PROMPT);
  console.log("AI response sucessfully received");

  // Extract the text response from AI
  let responseText = result.response.text();

  // Remove any markdown code block formatting (```json and ```)
  responseText = responseText
    .replace(/```json\n?/g, "")
    .replace(/```\n?/g, "")
    .trim();
  console.log(
    "Cleaned response (first 100 chars):",
    responseText.substring(0, 100),
  );

  // parsing or converting the ai response into javascript object
  const tripResp = JSON.parse(responseText);
  console.log("Parsing from text to JSON success");

  // Generate a unique document ID using current timestamp
  const docId = Date.now().toString();
  console.log("Generated a document ID: ", docId);

  //saving the NewTripData into the firebase firestore database
  console.log("Saving to Firebase Database");
  await setDoc(doc(db, "NewUserTrips", docId), {
    userEmail: user?.email || "unknown",

    destination: {
      name: tripData?.locationInfo?.name,
      full: tripData?.locationInfo,
    },
```

```
      budget: tripData?.budget?.title2,
      travellers: tripData?.travellers?.title,
      dates: {
        start: tripData?.startDate ? tripData.startDate.toISOString() : null,

        end: tripData?.endDate ? tripData.endDate.toISOString() : null,

        days: tripData?.totalNoOfDays,
      },

      tripPlan: tripResp,
      tripData: JSON.stringify(tripData),
      createdAt: Date.now(),
      docId: docId,
    });

    console.log("Successfully saved to Firebase");
    console.log("AI TRIP GENERATION COMPLETE");

    // 3.5 seconds of loading screen and then pushing to the mytrip tab
    setTimeout(() => {
      setIsLoading(false);
      router.push("../(tabs)/mytrip");
    }, 3500);
  } catch (error) {
    //hiding it when done
    setIsLoading(false);

    // Handle specific error types with appropriate user messages
    if (error.status === 429) {
      // User exceeded their daily AI request quota
      Alert.alert(
        "Quota Exceeded",
        "You've used all your daily AI requests. Please try again tomorrow.",
      );
    } else if (error.message.includes("JSON Parse")) {
      // AI returned invalid JSON format
      Alert.alert(
        "AI Response Error",
        "The AI returned an invalid format. Please try again.",
      );
    } else {
```

```
      // Generic error handler for all other errors
      Alert.alert("Error", "Failed to generate trip: " + error.message);
    }
  }
};

// Set navigation header options
useEffect(() => {
  navigation.setOptions({
    headerShown: false,
    headerTransparent: true,
    headerTitle: "",
  });
}, [navigation]);

return (
  <>
    <View style={{ flex: 1 }}>
      <View style={styles.background}>
        <TouchableOpacity
          onPress={() => router.back()}
          style={styles.backButton}
        >
          <Feather name="arrow-left" size={30} color="#000000ff" />
        </TouchableOpacity>

        <Text style={styles.tripReviewText}>Trip Review</Text>

        <Text style={styles.tripNameText}>Trip Name</Text>

        <TextInput
          style={styles.changeTripName}
          placeholder="Customize Name"
          placeholderTextColor="#aaaaaa7b"
          autoCapitalize="none"
        />

        <View style={styles.container}>
          <Text style={styles.destinationText}>Destination</Text>
          <TouchableOpacity
            onPress={() => router.back()}
            style={styles.changeAllOptions}
```

```
>
  <Octicons name="pencil" size={30} color="grey" style={{top:20}}/>
</TouchableOpacity>
<Text style={styles.destinationName}>
  {tripData?.locationInfo?.name || "Not selected"}
</Text>

<Octicons
  name="location"
  size={27}
  color="black"
  style={styles.locationIcon}
/>

<View style={styles.travellerContainer}>
  <Text style={styles.travellersText}>Travellers</Text>
  <Text style={styles.travellerCount}>
    {tripData?.travellers?.title || "Not selected"}
  </Text>
  <Octicons
    name="people"
    size={27}
    color="#0B5ED7"
    style={styles.travellersIcon}
  />
</View>

<View style={styles.dateContainer}>
  <Text style={styles.datesText}>Dates</Text>
  <Text style={styles.dateCount}>
    {tripData?.startDate && tripData?.endDate
      ? `${moment(tripData.startDate).format("DD MMM")} to ${moment(
          tripData.endDate,
        ).format("DD MMM")} (${tripData.totalNoOfDays || "?"} days)`
      : "Not selected"}
  </Text>
  <Feather
    name="calendar"
    size={27}
    color="black"
    style={styles.datesIcon}
  />
```

```
        </View>

        <View style={styles.costContainer}>
          <Text style={styles.costText}>Cost</Text>
          <Text style={styles.costTotal}>
            {tripData?.budget?.title2 && tripData?.budget?.desc2
              ? `${tripData.budget.title2}, ${tripData.budget.desc2}`
              : "Not selected"}
          </Text>
          <FontAwesome
            name="money"
            size={27}
            color="#0B5ED7"
            style={styles.costIcon}
          />
        </View>
      </View>

      <View style={styles.continueButton}>
        <TouchableOpacity onPress={handleContinue}>
          <Text style={styles.continueText}>Continue to Trip</Text>
        </TouchableOpacity>
      </View>
    </View>
  </View>

  {/* Loading Modal */}
  <Modal
    transparent={true}
    animationType="fade"
    visible={isLoading}
    statusBarTranslucent={true}
  >
    <BlurView
      intensity={10}
      tint="light"
      style={[StyleSheet.absoluteFill, { borderRadius: 24 }]}
    />
    <View style={styles.loadingOverlay}>
      <View style={styles.loadingCard}>
        <TouchableOpacity style={styles.xButton} onPress={handleCancel}>
          <MaterialCommunityIcons
```

```
              name="exit-to-app"
              size={24}
              color="#0B5ED7"
            />
          </TouchableOpacity>

          <Image
            source={require("../../assets/images/TripReview/loadingScreen.gif")}
            style={styles.loadingGif}
            resizeMode="contain"
          />

          <Text style={styles.generatingText}>
            Generating the best trip for you...
          </Text>

          <ActivityIndicator
            size="large"
            color={Colors.BLUE}
            style={{ marginTop: 20 }}
          />
        </View>
      </View>
    </Modal>
  </>
 );
}

const styles = StyleSheet.create({
 background: {
   backgroundColor: "#fff",
   height: 1000,
 },
 container: {
   marginTop: -50,
 },
 map: {
   position: "absolute",
   width: 700,
   height: 700,
   zIndex: 1,
   opacity: 0.2,
```

```
  top: 150,
  left: -40,
  transform: [{ rotate: "20deg" }],
},
tripReviewText: {
  fontFamily: "outfit-bold",
  color: Colors.BLUE,
  fontSize: 35,
  marginTop: 100,
  top: -25,
  textAlign: "center",
  zIndex: 2,
},
tripNameText: {
  fontFamily: "outfit-medium",
  fontSize: 18,
  color: Colors.BLACK,
  paddingLeft: 20,
  paddingTop: 10,
  marginBottom: 10,
  zIndex: 2,
},
backButton: {
  position: "absolute",
  paddingLeft: 20,
  paddingTop: 80,
  zIndex: 3,
},
changeTripName: {
  fontFamily: "outfit-medium",
  backgroundColor: Colors.WHITE,
  marginTop: 6,
  borderColor: Colors.BLUE,
  borderWidth: 1.2,
  borderRadius: 30,
  paddingHorizontal: 20,
  fontSize: 16,
  shadowColor: Colors.BLUE,
  shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.1,
  shadowRadius: 4,
  elevation: 3,
```

```
  width: 350,
  height: 60,
  left: 20,
  zIndex: 2,
},
destinationText: {
  fontFamily: "outfit-medium",
  color: Colors.GREY,
  fontSize: 20,
  top: 105,
  left: 80,
  zIndex: 2,
},
destinationName: {
  fontFamily: "outfit-bold",
  color: Colors.BLUE,
  fontSize: 20,
  top: 78,
  left: 80,
  zIndex: 2,
},
locationIcon: {
  paddingLeft: 35,
  paddingTop: 40,
  zIndex: 2,
},
travellerContainer: {
  marginTop: -70,
},
travellersText: {
  fontFamily: "outfit-medium",
  color: Colors.GREY,
  fontSize: 20,
  top: 115,
  left: 80,
  zIndex: 2,
},
travellerCount: {
  fontFamily: "outfit-bold",
  fontSize: 20,
  top: 115,
  left: 80,
```

```
  zIndex: 2,
},
travellersIcon: {
  paddingLeft: 35,
  paddingTop: 80,
  zIndex: 2,
},
dateContainer: {
  marginTop: -70,
},
datesText: {
  color: Colors.GREY,
  fontFamily: "outfit-medium",
  fontSize: 20,
  top: 115,
  left: 80,
  zIndex: 2,
},
dateCount: {
  fontFamily: "outfit-bold",
  color: Colors.BLUE,
  fontSize: 20,
  top: 115,
  left: 80,
  zIndex: 2,
},
datesIcon: {
  paddingLeft: 35,
  paddingTop: 80,
  zIndex: 2,
},
costContainer: {
  marginTop: -70,
  zIndex: 2,
},
costText: {
  color: Colors.GREY,
  fontFamily: "outfit-medium",
  fontSize: 20,
  top: 115,
  left: 80,
  zIndex: 2,
```

```
},
costTotal: {
  fontFamily: "outfit-bold",
  fontSize: 20,
  top: 115,
  left: 80,
  zIndex: 2,
},
costIcon: {
  paddingLeft: 35,
  paddingTop: 80,
  zIndex: 2,
},
changeAllOptions: {
  paddingLeft: 25,
  top: 440,
  zIndex: 2,
},
continueText: {
  fontFamily: "outfit-medium",
  fontSize: 25,
  color: Colors.WHITE,
  textAlign: "center",
  marginTop: 15,
  zIndex: 2,
},
continueButton: {
  marginTop: 70,
  marginLeft: 70,
  height: 60,
  width: 300,
  backgroundColor: Colors.BLUE,
  borderRadius: 40,
  zIndex: 2,
},
loadingOverlay: {
  flex: 1,
  backgroundColor: "rgba(0, 0, 0, 0.65)",
  justifyContent: "center",
  alignItems: "center",
},
loadingCard: {
```

```
    backgroundColor: "#FFFFFF",
    borderRadius: 24,
    padding: 40,
    alignItems: "center",
    width: "85%",
    maxWidth: 360,
    shadowColor: "#000",
    shadowOffset: { width: 0, height: 10 },
    shadowOpacity: 0.3,
    shadowRadius: 20,
    elevation: 15,
  },
  xButton: {
    position: "absolute",
    top: 16,
    right: 16,
    padding: 8,
    zIndex: 10,
  },
  loadingGif: {
    width: 330,
    height: 200,
    marginBottom: 20,
  },
  generatingText: {
    fontFamily: "outfit-bold",
    fontSize: 22,
    color: Colors.BLUE,
    textAlign: "center",
    marginBottom: 16,
  },
});
```

7. Itinerary

```
import { Colors } from "@/app-example/constants/theme";
import { Image, StyleSheet, Text, View } from "react-native";

export default function PlannedTrips({ itineraryInfo }) {
  if (!itineraryInfo || typeof itineraryInfo !== "object") {
    return (
      <View style={{ padding: 20 }}>
```

```
      <Text>No itinerary available</Text>
    </View>
  );
}

return (
  <View style={styles.container}>
    {Object.entries(itineraryInfo).map(([day, details], dayIndex) => (
      <View key={day} style={styles.daySection}>
        {/* DAY HEADER */}
        <View style={styles.dayHeaderRow}>
          <View style={styles.dayBadge}>
            <Text style={styles.dayBadgeNumber}>{dayIndex + 1}</Text>
          </View>
          <View style={styles.dayHeaderLine} />
          <View style={styles.dayHeaderLine} />
        </View>

        {/* PLACE CARDS */}
        {(details?.plan || []).map((place, index) => (
          <View key={index} style={styles.card}>
            {/* DESTINATION STOPS */}
            <View style={styles.indexPill}>
              <Text style={styles.indexPillText}>Stop {index + 1}</Text>
            </View>

            {/* IMAGE */}
            <Image
              style={styles.image}

source={require("./../../assets/images/MyTripsImages/placeholder-1-e1533569576673.webp
")}
            />

            {/* CARD BODY */}
            <View style={styles.cardBody}>
              {/* PLACE NAME */}
              <Text style={styles.placeNames}>
                {place?.placeName || "Unknown place"}
              </Text>

              {/* DESCRIPTION */}
```

```
              <Text style={styles.description}>
                {place?.placeDetails || "No details available"}
              </Text>

              {/* DIVIDER */}
              <View style={styles.divider} />

              {/* COST + DURATION ROW */}
              <View style={styles.metaRow}>
                <View style={styles.metaChip}>
                  <Text style={styles.metaIcon}>🎫</Text>
                  <Text style={styles.metaLabel}>Cost</Text>
                  <Text style={styles.metaValue}>
                    {place?.ticketPricing || "-"}
                  </Text>
                </View>

                <View style={styles.metaChipDivider} />

                <View style={styles.metaChip}>
                  <Text style={styles.metaIcon}>🕒</Text>
                  <Text style={styles.metaLabel}>Duration</Text>
                  <Text style={styles.metaValue}>
                    {place?.timeToTravel || "-"}
                  </Text>
                </View>
              </View>
            </View>
          ))}
        </View>
      ))}
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    paddingBottom: 30,
  },

  daySection: {
```

```
  marginTop: 30,
},

dayHeaderRow: {
  flexDirection: "row",
  alignItems: "center",
  marginBottom: 16,
  paddingHorizontal: 4,
},

dayBadge: {
  width: 34,
  height: 34,
  borderRadius: 17,
  backgroundColor: Colors.BLUE,
  justifyContent: "center",
  alignItems: "center",
  marginRight: 10,
},

dayBadgeNumber: {
  color: "#fff",
  fontSize: 14,
  fontFamily: "outfit-bold",
},

dayHeaderLine: {
  flex: 1,
  height: 1,
  backgroundColor: "#E5EAF0",
},

card: {
  backgroundColor: "#fff",
  borderRadius: 24,
  marginBottom: 18,
  overflow: "hidden",
  elevation: 5,
},

indexPill: {
  position: "absolute",
```

```
  top: 14,
  left: 14,
  zIndex: 10,
  backgroundColor: Colors.BLUE,
  paddingHorizontal: 12,
  paddingVertical: 5,
  borderRadius: 99,
},

indexPillText: {
  color: "#fff",
  fontFamily: "outfit-medium",
  fontSize: 12,
},

image: {
  width: "100%",
  height: 180,
},

cardBody: {
  padding: 18,
},

placeNames: {
  fontFamily: "outfit-bold",
  fontSize: 20,
},

description: {
  fontFamily: "outfit-medium",
  fontSize: 14,
  color: "#6B7280",
},

divider: {
  height: 1,
  backgroundColor: "#F0F3F7",
  marginVertical: 14,
},

metaRow: {
```

```
    flexDirection: "row",
  },

  metaChip: {
    flex: 1,
    alignItems: "center",
  },

  metaIcon: {
    fontSize: 18,
  },

  metaLabel: {
    fontSize: 11,
    color: "#9CA3AF",
  },

  metaValue: {
    fontSize: 13,
    color: Colors.BLUE,
  },

  metaChipDivider: {
    width: 1,
    height: 36,
    backgroundColor: "#ddd",
  },
});
```

8. First Trip

```
import { Colors } from "@/app-example/constants/theme";
import Octicons from "@expo/vector-icons/Octicons";
import { useRouter } from "expo-router";
import moment from "moment";
import { Image, StyleSheet, Text, TouchableOpacity, View } from "react-native";
import UserTripCard from "./UserTripCard";
//BIG ONE
export default function UserTripList({ userTrips }) {
  const router = useRouter();

  //parsing unfiltered/raw data into json format
  const LatestTrip = JSON.parse(userTrips[0].tripData);
```

```
 console.log("photoRef:", LatestTrip?.locationInfo?.photoRef);
 console.log(
   "Full URL:",
   "https://places.googleapis.com/v1/" +
     LatestTrip?.locationInfo?.photoRef +
     "/media?key=" +
     process.env.EXPO_PUBLIC_GOOGLE_MAP_KEY,
 );

 const estimatedPriceRaw =
   userTrips[0]?.tripPlan?.travelPlan?.flightDetails?.estimatedPrice;

 //extracting the numerals from the raw data
 const estimatedPrice = estimatedPriceRaw?.match(/\d+/g);

 const formattedPrice = estimatedPrice
   ? `$${estimatedPrice[0]} - $${estimatedPrice[1]}`
   : "N/A";

 return (
   <View>
     <View
       style={{
         margin: 20,
         width: "100%",
       }}
     >
       <View style={styles.containerBorder}>
         <View>
           {LatestTrip?.locationInfo?.photoRef ? (
             <Image
               source={{
                 uri:

"https://maps.googleapis.com/maps/api/place/photo?maxwidth=400&photo_reference=" +
                   LatestTrip.locationInfo?.photoRef +
                   "&key=" +
                   process.env.EXPO_PUBLIC_GOOGLE_MAP_KEY,
               }}
               style={{ width: "100%", height: 200, borderRadius: 20 }}
               resizeMode="cover"
             />
```

```
          ) : (
            <Image

source={require("./../../assets/images/MyTripsImages/placeholder-1-e1533569576673.webp
")}
              style={styles.tripImage}
              resizeMode="cover"
              margin="0"
            />
          )}
        </View>
        <View style={{ margin: 15, marginTop: 5 }}>
          <Octicons
            name="location"
            size={20}
            color="black"
            style={styles.locationIcon}
          />
          <Text style={styles.tripNameText}>
            {/* Question Mark means that it is optional so there would be less errors
*/}
            {userTrips[0]?.tripPlan?.travelPlan?.location}
          </Text>
          <Text style={styles.tripDateText}>
            {moment(LatestTrip.startDate).format("DD of MMM, yyyy")}
          </Text>

          <View style={styles.row}>
            <Text style={styles.tripTravellersText}>
              {LatestTrip.travellers.title}
            </Text>
            <Text style={styles.formattedPrice}>{formattedPrice}</Text>
          </View>
          <TouchableOpacity
            style={styles.buttonContainer}
            activeOpacity={0.8}
            onPress={() =>
              router.push({
                pathname: "/trip-detail",
                params: { trip: JSON.stringify(userTrips[0]) },
              })
            }
```

```
          >
            <Text style={styles.buttonProceed}>Proceed</Text>
          </TouchableOpacity>
        </View>
      </View>
      <Text style={styles.allTrips}>All Trips</Text>
      {/* usertrips is an array, map allows it to go through each element and return
its index */}
      {userTrips.map((trip, index) => (
        <UserTripCard trip={trip} key={index} />
      ))}
    </View>
  </View>
 );
}

const styles = StyleSheet.create({
 containerBorder: {
  borderColor: Colors.GREY,
  borderWidth: 1,
  borderRadius: 20,
  width: 355,
  overflow: "hidden",
  padding: 10,
 },
 ongoingText: {
  fontFamily: "outfit-bold",
  color: Colors.BLUE,
  fontSize: 20,
  paddingBottom: 20,
 },

 tripImage: {
  width: "100%",
  height: 200,
  borderRadius: 20,
 },

 tripNameText: {
  fontFamily: "outfit-medium",
  fontSize: 20,
  color: Colors.BLACK,
```

```
  paddingLeft: 25,
},

locationIcon: {
  top: 20,
  color: Colors.BLUE,
},

tripDateText: {
  fontFamily: "outfit-medium",
  fontSize: 15,
  color: Colors.DARKGREY,
  marginTop: 5,
},

tripTravellersText: {
  fontFamily: "outfit-medium",
  fontSize: 15,
  color: Colors.DARKGREY,
  marginTop: 5,
},

row: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  marginTop: 1,
},
formattedPrice: {
  fontFamily: "outfit-medium",
  fontSize: 20,
  color: Colors.BLUE,
},

buttonContainer: {
  justifyContent: "center",
  alignItems: "center",
  marginTop: 15,
  marginBottom: 5,
  marginLeft: -15,
  backgroundColor: Colors.BLUE,
  height: 50,
```

```
    width: 335,
    borderRadius: 20,
  },

  buttonProceed: {
    fontFamily: "outfit-bold",
    alignContent: "center",
    textAlign: "center",
    color: Colors.WHITE,
  },

  allTrips: {
    fontFamily: "outfit-bold",
    color: Colors.BLUE,
    fontSize: 20,
    marginTop: 15,
  },
});
```

9. All Trips

```
import { Colors } from "@/app-example/constants/theme";
import Octicons from "@expo/vector-icons/Octicons";
import { useRouter } from "expo-router";
import moment from "moment";
import { Image, StyleSheet, Text, TouchableOpacity, View } from "react-native";
import UserTripCard from "./UserTripCard";
//BIG ONE
export default function UserTripList({ userTrips }) {
  const router = useRouter();

  //parsing unfiltered/raw data into json format
  const LatestTrip = JSON.parse(userTrips[0].tripData);
  console.log("photoRef:", LatestTrip?.locationInfo?.photoRef);
  console.log(
    "Full URL:",
    "https://places.googleapis.com/v1/" +
      LatestTrip?.locationInfo?.photoRef +
      "/media?key=" +
      process.env.EXPO_PUBLIC_GOOGLE_MAP_KEY,
  );
```

```
 const estimatedPriceRaw =
   userTrips[0]?.tripPlan?.travelPlan?.flightDetails?.estimatedPrice;

 //extracting the numerals from the raw data
 const estimatedPrice = estimatedPriceRaw?.match(/\d+/g);

 const formattedPrice = estimatedPrice
   ? `$${estimatedPrice[0]} - $${estimatedPrice[1]}`
   : "N/A";

 return (
   <View>
     <View
       style={{
         margin: 20,
         width: "100%",
       }}
     >
       <View style={styles.containerBorder}>
         <View>
           {LatestTrip?.locationInfo?.photoRef ? (
             <Image
               source={{
                 uri:

"https://maps.googleapis.com/maps/api/place/photo?maxwidth=400&photo_reference=" +
                   LatestTrip.locationInfo?.photoRef +
                   "&key=" +
                   process.env.EXPO_PUBLIC_GOOGLE_MAP_KEY,
               }}
               style={{ width: "100%", height: 200, borderRadius: 20 }}
               resizeMode="cover"
             />
           ) : (
             <Image

source={require("./../../assets/images/MyTripsImages/placeholder-1-e1533569576673.webp
")}
               style={styles.tripImage}
               resizeMode="cover"
               margin="0"
             />
```

```
          )}
        </View>
        <View style={{ margin: 15, marginTop: 5 }}>
          <Octicons
            name="location"
            size={20}
            color="black"
            style={styles.locationIcon}
          />
          <Text style={styles.tripNameText}>
            {/* Question Mark means that it is optional so there would be less errors
*/}
            {userTrips[0]?.tripPlan?.travelPlan?.location}
          </Text>
          <Text style={styles.tripDateText}>
            {moment(LatestTrip.startDate).format("DD of MMM, yyyy")}
          </Text>

          <View style={styles.row}>
            <Text style={styles.tripTravellersText}>
              {LatestTrip.travellers.title}
            </Text>
            <Text style={styles.formattedPrice}>{formattedPrice}</Text>
          </View>
          <TouchableOpacity
            style={styles.buttonContainer}
            activeOpacity={0.8}
            onPress={() =>
              router.push({
                pathname: "/trip-detail",
                params: { trip: JSON.stringify(userTrips[0]) },
              })
            }
          >
            <Text style={styles.buttonProceed}>Proceed</Text>
          </TouchableOpacity>
        </View>
      </View>
      <Text style={styles.allTrips}>All Trips</Text>
      {/* usertrips is an array, map allows it to go through each element and return
its index */}
      {userTrips.map((trip, index) => (
```

```
          <UserTripCard trip={trip} key={index} />
        ))}
      </View>
    </View>
  );
}

const styles = StyleSheet.create({
  containerBorder: {
    borderColor: Colors.GREY,
    borderWidth: 1,
    borderRadius: 20,
    width: 355,
    overflow: "hidden",
    padding: 10,
  },
  ongoingText: {
    fontFamily: "outfit-bold",
    color: Colors.BLUE,
    fontSize: 20,
    paddingBottom: 20,
  },

  tripImage: {
    width: "100%",
    height: 200,
    borderRadius: 20,
  },

  tripNameText: {
    fontFamily: "outfit-medium",
    fontSize: 20,
    color: Colors.BLACK,
    paddingLeft: 25,
  },

  locationIcon: {
    top: 20,
    color: Colors.BLUE,
  },

  tripDateText: {
```

```
  fontFamily: "outfit-medium",
  fontSize: 15,
  color: Colors.DARKGREY,
  marginTop: 5,
},

tripTravellersText: {
  fontFamily: "outfit-medium",
  fontSize: 15,
  color: Colors.DARKGREY,
  marginTop: 5,
},

row: {
  flexDirection: "row",
  justifyContent: "space-between",
  alignItems: "center",
  marginTop: 1,
},
formattedPrice: {
  fontFamily: "outfit-medium",
  fontSize: 20,
  color: Colors.BLUE,
},

buttonContainer: {
  justifyContent: "center",
  alignItems: "center",
  marginTop: 15,
  marginBottom: 5,
  marginLeft: -15,
  backgroundColor: Colors.BLUE,
  height: 50,
  width: 335,
  borderRadius: 20,
},

buttonProceed: {
  fontFamily: "outfit-bold",
  alignContent: "center",
  textAlign: "center",
  color: Colors.WHITE,
```

```
 },

 allTrips: {
   fontFamily: "outfit-bold",
   color: Colors.BLUE,
   fontSize: 20,
   marginTop: 15,
 },
});
```